1. AI-POWERED DRONES FOR EMERGENCY MEDICINE DELIVERY

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty.

Greedy Best-First Search is a search algorithm that selects the path that appears closest to the destination based on a heuristic value. In this scenario, the drone uses straight-line distance to estimate how close it is to the location where emergency medicines must be delivered.

For example, if the drone has two possible routes, it will generally select the route that appears shorter according to the heuristic.

Strengths:

- It can find a route quickly because it focuses on the estimated distance to the destination.
- It is useful when time is limited and emergency medicines must be delivered quickly.
- It requires less exploration than some other search methods.

Weaknesses:

- It may select a shorter route that is actually blocked by floods or damaged roads.
- It does not properly consider the total travel cost.
- It may produce a longer or unsafe route when weather conditions change.

Therefore, Greedy Best-First Search is fast, but it may not always provide the safest or most efficient route.

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic.

A* Search is an informed search algorithm that considers both the actual cost of reaching the current location and the estimated cost of reaching the destination.

It uses the following formula:

f(n) = g(n) + h(n)

Where:

- g(n) = Actual cost from the starting point to the current location.
- h(n) = Estimated cost from the current location to the destination.
- f(n) = Total estimated cost of the complete route.

In this scenario, the drone considers the distance already travelled, weather conditions, terrain difficulties and the estimated distance remaining.

For example, a route may be shorter in distance but have heavy rainfall or dangerous obstacles. A* can consider the increased travel cost and select another route with a lower estimated total cost.

This makes A* more effective than Greedy Best-First Search because it considers both the current travel cost and the remaining distance.

iii. Analyze the impact of heuristic accuracy on both algorithms.

The accuracy of the heuristic plays an important role in the performance of both algorithms.

In Greedy Best-First Search, the heuristic directly determines which location is explored next. If the straight-line distance is misleading, the drone may choose a route that is blocked or dangerous.

In A* Search, the heuristic helps estimate the remaining travel cost. An accurate heuristic helps the algorithm identify better routes and improves search efficiency.

For example, if a flood blocks the shortest route, a simple distance-based heuristic may not reflect the actual difficulty of reaching the destination.

Therefore, a more realistic heuristic that considers terrain, weather and possible obstacles can improve the performance of both algorithms.

iv. Discuss how dynamic changes (blocked paths) affect search performance.

Dynamic changes can make route planning difficult because the environment does not remain the same throughout the journey.

For example, a road that was available when the drone started travelling may become blocked due to flooding or strong winds.

When this happens:

- The drone must identify the blocked path.
- The previous route may no longer be suitable.
- The search algorithm must consider alternative routes.
- Additional time and computational resources may be required.

A* may need to repeat part of its search when the environment changes. Using updated information and replanning techniques can help the drone find a new route.

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety.

I would recommend **A* Search with real-time route updates** for this situation.

The main reason is that emergency medicine delivery requires a balance between speed, travel cost and safety.

A* considers both the actual travel cost and the estimated remaining cost. It can therefore identify a better route than an algorithm that only considers distance.

The drone can also update its route whenever new information about blocked paths or weather conditions becomes available.

However, safety should always be given priority. The drone should avoid dangerous routes even if they appear shorter.

Thus, A* with frequent route updates is a suitable choice for delivering emergency medicines in a changing environment.

2. SMART CITY TRAFFIC SIGNAL OPTIMIZATION

Formulate this as an optimization problem and explain why traditional search is inefficient.

A smart city traffic management system aims to reduce traffic problems by automatically adjusting traffic signal timings at different intersections.

The main objectives are:

- Reducing the total waiting time of vehicles.
- Minimizing fuel consumption.
- Reducing traffic congestion.
- Improving the movement of emergency vehicles.
- Making better use of available road capacity.

The problem can be formulated as an optimization problem.

Objective Function:

Minimize:

Total Cost = W1(Total Waiting Time) + W2(Fuel Consumption) + W3(Traffic Congestion)

Here, W1, W2 and W3 represent the importance given to each objective.

The decision variables include the green signal duration, red signal duration and timing sequence at each intersection.

Traditional search methods are inefficient because the number of possible signal combinations becomes extremely large when hundreds of intersections are involved.

Also, traffic conditions change continuously, so a solution that works at one moment may not work a few minutes later.

Therefore, optimization techniques are needed to find effective solutions within a reasonable amount of time.

i. Explain how Hill Climbing would work in this scenario and identify its limitations.

Hill Climbing is a local search algorithm that starts with an initial solution and continuously tries to improve it by selecting a better neighbouring solution.

In this scenario, the system first creates an initial traffic signal schedule.

It then changes the signal timings at different intersections and checks whether the changes reduce waiting time, fuel consumption or congestion.

If a new schedule gives a better result, the system accepts that schedule and continues searching for improvements.

For example, if increasing the green signal time at a busy intersection reduces vehicle waiting time, the system may keep that change.

Limitations:

- It may get stuck in a local optimum.
- It may fail to find the best overall traffic schedule.
- It can struggle when many intersections are connected.
- It may not perform well when traffic conditions change suddenly.

Therefore, Hill Climbing is simple and useful for local improvements, but it may not always find the best solution.

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation.

Hill Climbing faces several problems while searching for an optimal solution.

1. Local Maxima:

A local maximum is a solution that appears to be the best among its neighbouring solutions, but it is not the best overall solution.

In traffic management, a signal schedule may improve traffic at one intersection but increase congestion in nearby areas.

The system may consider the current schedule satisfactory and stop searching.

2. Plateaus:

A plateau is an area where several neighbouring solutions have almost the same evaluation value.

For example, changing the green signal time by a few seconds may not produce any noticeable improvement in traffic flow.

As a result, the algorithm may struggle to decide which direction to move.

3. Ridges:

A ridge occurs when reaching a better solution requires a combination of changes rather than a single simple change.

For example, improving traffic flow may require changing signal timings at several connected intersections together.

Changing only one intersection may not produce any improvement.

These problems can prevent Hill Climbing from reaching the best possible traffic schedule.

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations.

Simulated Annealing and Genetic Algorithms are optimization techniques that help overcome some of the limitations of Hill Climbing.

Simulated Annealing:

Simulated Annealing is inspired by the process of heating and slowly cooling metals.

Unlike Hill Climbing, it sometimes accepts a worse solution to avoid getting stuck in a local optimum.

Initially, it allows more exploration. As the temperature decreases, it becomes more selective and focuses on improving the solution.

In traffic management, this allows the system to explore different signal combinations before settling on a better schedule.

Genetic Algorithm:

Genetic Algorithm is inspired by natural selection and evolution.

It works with a population of possible solutions rather than considering only one solution.

Its main operations are:

- Selection: Choosing better traffic schedules.
- Crossover: Combining parts of two schedules to create new schedules.
- Mutation: Making small random changes to signal timings.
- Fitness Evaluation: Measuring how effectively each schedule reduces traffic problems.

For example, two good traffic schedules can be combined to create a new schedule that performs better across multiple intersections.

Both techniques help explore a larger search space and reduce the possibility of getting stuck in a local optimum.

iv. Recommend the best approach and justify your choice based on scalability and adaptability.

I would recommend a **Genetic Algorithm combined with local optimization** for this smart city traffic management system.

A Genetic Algorithm can explore many possible traffic schedules at the same time. It is suitable for large optimization problems involving hundreds of intersections.

It can also adapt to changing traffic conditions by generating and evaluating new schedules.

Local optimization can further improve the solutions produced by the Genetic Algorithm.

The system should continuously collect traffic information and update its optimization process.

Therefore, a Genetic Algorithm with local optimization provides a practical approach for handling large traffic networks and changing traffic conditions.

3. AUTONOMOUS MARS ROVER EXPLORATION

i. Explain why this problem requires an online search agent instead of offline search.

An autonomous Mars rover operates in an unfamiliar environment where a complete map of the surroundings is not available.

An offline search agent requires sufficient information about the environment before planning its actions. However, the rover cannot know every obstacle, crater or dangerous area before starting its journey.

An online search agent makes decisions while interacting with the environment.

For example, when the rover moves forward, it may discover a large rock or a crater that was not previously known.

It can then update its information and select a different path.

The main advantages of online search are:

- It works with incomplete information.
- It learns about the environment while moving.
- It can change its decisions whenever new information becomes available.
- It helps the rover respond to unexpected obstacles.

Therefore, online search is more suitable for Mars exploration because the rover must make decisions while discovering unknown terrain.

ii. Analyze the challenges of partial observability and dynamic environments.

The Mars rover faces several challenges because it cannot observe the entire environment at once.

Partial Observability:

Partial observability means that the rover has access to only limited information about its surroundings.

For example, its camera or sensors may detect a rock nearby but may not reveal a deep crater behind it.

This makes decision-making difficult because the rover cannot always predict what lies ahead.

Dynamic Environment:

The rover may also experience changes in environmental conditions, such as dust storms, poor visibility and unexpected terrain difficulties.

These conditions can affect its movement and sensing ability.

The major challenges include:

- Limited sensor information.
- Unknown obstacles and dangerous areas.
- Uncertainty about the condition of the terrain.

- Communication delays with Earth.
- Limited battery power and processing resources.

The rover must carefully evaluate its surroundings before performing any action.

iii. Describe how the rover builds and updates its internal model.

An internal model is the information maintained by the rover about its surroundings and the possible actions it can perform.

The rover builds this model using information collected from its cameras, sensors and other onboard instruments.

The process includes:

1. Collecting Information:

The rover observes its surroundings using cameras and sensors.

2. Identifying Objects:

It identifies rocks, craters, slopes and other obstacles.

3. Creating a Map:

The rover creates a map of the explored area and records safe and dangerous locations.

4. Updating Information:

Whenever new information is received, the rover updates its internal map.

5. Planning Actions:

Using the updated information, it decides which direction to move and which areas to avoid.

For example, if the rover discovers a deep crater in its planned route, it updates its map and searches for another safe path.

This continuous process helps the rover make better decisions during exploration.

iv. Compare online search with uninformed and informed search strategies.

Online Search:

Online search makes decisions while the agent interacts with the environment. It is useful when the complete environment is unknown.

Uninformed Search:

Uninformed search does not use additional knowledge or heuristic information to guide the search.

Examples include Breadth-First Search and Depth-First Search.

These methods explore possible paths without using an estimate of how close the destination is.

Informed Search:

Informed search uses heuristic information to guide the search towards a goal.

Examples include Greedy Best-First Search and A* Search.

These algorithms use additional knowledge to select promising paths.

Comparison:

- Online search is suitable for unknown environments.
- Uninformed search works without heuristic guidance.
- Informed search uses estimates to improve decision-making.
- Online search can update its knowledge while exploring.
- Informed and uninformed search can be used with available environmental information, but traditional offline versions generally assume the search model is sufficiently known.

For the Mars rover, an online approach is important because the environment is not completely known in advance.

v. Justify the most suitable approach considering uncertainty, adaptability, and safety.

I would recommend an **online informed search approach with continuous environmental updates** for the Mars rover.

The rover must explore unknown locations while avoiding dangerous obstacles.

An online informed approach allows it to collect information, update its internal model and use heuristic guidance to select promising paths.

For example, if the rover identifies a crater in front of it, it can update its map and search for a safer alternative route.

Safety should always be the main priority. The rover should also consider battery usage, terrain conditions and the risk of damaging its equipment.

Thus, an online informed search approach provides a suitable balance between exploration, adaptability and safety.

4. UNIVERSITY EXAM TIMETABLE GENERATION USING CSP

Formulate the problem as a Constraint Satisfaction Problem (CSP).

A university needs to prepare an examination timetable for different courses and students.

The main challenge is to schedule all examinations without creating conflicts.

A Constraint Satisfaction Problem (CSP) is a problem-solving technique in which variables are assigned values while satisfying a set of given constraints.

In this scenario, the variables represent examinations, the domains represent available time slots and examination halls, and the constraints represent the rules that must be followed.

i. Clearly define variables, domains, and constraints with explanation.

1. Variables:

Variables represent the examinations that must be scheduled.

For example:

- Exam 1: Artificial Intelligence.
- Exam 2: Database Management.
- Exam 3: Python Programming.
- Exam 4: Computer Networks.

Each examination must be assigned a suitable time slot and examination hall.

2. Domains:

Domains represent the possible values that can be assigned to the variables.

For example:

- Morning examination slot.
- Afternoon examination slot.
- Available examination halls.
- Different examination dates.

The system selects suitable values from these domains.

3. Constraints:

Constraints are the rules that must be satisfied while creating the timetable.

Important constraints include:

- No student should have two examinations at the same time.
- Two examinations cannot use the same hall at the same time.
- The number of students must not exceed the hall capacity.
- Faculty members must be available during the scheduled examination.
- Certain subjects must be scheduled before other subjects.

- Special accommodations must be provided for students who need them.

By satisfying these conditions, the system can generate a valid examination timetable.

ii. Explain how backtracking search works in this scenario.

Backtracking Search is a method used to solve CSP problems by assigning values to variables one at a time.

In this scenario, the system first selects an examination and assigns an available time slot and hall.

After assigning a value, it checks whether the assignment satisfies all the constraints.

If the assignment is valid, the system moves to the next examination.

However, if a conflict occurs, the system goes back to the previous assignment and tries another available option.

Example:

Suppose Artificial Intelligence and Database Management have common students.

If both examinations are scheduled at 10:00 AM, the system identifies a conflict.

It then goes back and assigns one of the examinations to another available time slot.

This process continues until all examinations are scheduled successfully or the system determines that the current combination has no valid solution.

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency.

Constraint propagation helps reduce unnecessary searching by removing values that cannot satisfy the constraints.

Forward checking is one of the techniques used for this purpose.

Whenever the system assigns a time slot to an examination, it checks the domains of the remaining examinations.

If another examination cannot use the same time slot because of overlapping students or hall availability, that value is removed from its domain.

Example:

Suppose an Artificial Intelligence examination is assigned to Monday morning.

If Database Management has common students with Artificial Intelligence, Monday morning is removed from its available time slots.

This prevents the system from choosing an invalid assignment later.

The main benefits of forward checking are:

- Reduces unnecessary searches.
- Detects conflicts earlier.
- Saves processing time.
- Improves the efficiency of backtracking.

- Helps find a valid timetable faster.

iv. Analyze real-world challenges and justify the best strategy for scalability.

Generating a timetable for a large university is difficult because there may be hundreds of courses, thousands of students and limited examination halls.

Some real-world challenges include:

- A large number of examinations.
- Students enrolled in multiple courses.
- Limited hall capacity.
- Faculty availability.
- Special accommodation requirements.
- Changes in examination schedules.
- Increasing complexity as the number of courses grows.

I would recommend using **Backtracking Search with constraint propagation and suitable variable-ordering techniques**.

The Minimum Remaining Values (MRV) technique can select the examination with the fewest available options first.

Forward checking can remove conflicting options immediately after an assignment.

These techniques reduce unnecessary searching and improve the overall efficiency of the timetable generation system.

Therefore, combining backtracking, MRV and forward checking is a suitable approach for generating examination timetables while satisfying multiple constraints.

5. STRATEGIC GAME AI FOR REAL-TIME DECISION MAKING (MINIMAX & ALPHA-BETA PRUNING)

i. Explain how the Minimax algorithm models this problem.

Minimax is a decision-making algorithm used in competitive games where two players have opposing objectives.

It assumes that one player tries to maximize the score while the other player tries to minimize it.

In this scenario, the AI opponent acts as the maximizing player, while the human opponent acts as the minimizing player.

The algorithm explores possible future moves and evaluates the resulting game states.

For example, the AI considers its possible moves, predicts the human player's responses and selects the move that provides the best possible outcome against the opponent's strongest response.

The main idea is to choose a move that gives the AI the best result even when the opponent plays intelligently.

Minimax is useful because it helps the AI make strategic decisions rather than choosing moves randomly.

ii. Analyze the computational challenges of large game trees.

In a real-time strategy game, each player may have many possible moves.

Every possible move creates additional game states, and each new state may produce several more possibilities.

This creates a very large search tree.

The major computational challenges include:

- A large number of possible moves.
- Increasing search depth.
- High memory and processing requirements.
- Limited time for making decisions.
- Difficulty in evaluating complex game situations.

For example, if each game state has 10 possible moves and the algorithm searches 5 levels deep, it may need to examine up to 100,000 possible states.

This makes traditional Minimax expensive for complex games.

Therefore, techniques are needed to reduce the number of states that must be explored.

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning.

Alpha-Beta Pruning is an optimization technique used with the Minimax algorithm.

It reduces the number of game states that need to be examined without changing the final Minimax decision, provided the same search depth and evaluation are used.

It works by maintaining two values:

Alpha (α): The best value that the maximizing player can guarantee so far.

Beta (β): The best value that the minimizing player can guarantee so far.

Whenever the algorithm finds a situation where:

Alpha ≥ Beta

It stops exploring the remaining branches because they cannot improve the final decision.

Example:

Suppose the AI is examining two possible moves.

The first move gives an evaluation value of 8.

While examining the second move, the algorithm discovers that the opponent can force a result of 5 or lower.

Since the AI already has a better option with a value of 8, there is no need to explore further branches that cannot improve the decision.

Those branches are pruned.

Advantages of Alpha-Beta Pruning:

- Reduces the number of nodes explored.
- Saves processing time.
- Allows deeper searches.
- Improves decision-making efficiency.
- Produces the same Minimax result when applied correctly.

Therefore, Alpha-Beta Pruning makes Minimax more practical for games with large search trees.

iv. Design an evaluation function and justify the factors considered.

An evaluation function is used to estimate how good a particular game state is when the algorithm cannot search all the way to the end of the game.

For a real-time strategy game, the evaluation function can consider the following factors:

Evaluation Function:

Evaluation Score = (3 × Resource Advantage) + (5 × Unit Strength) + (4 × Territory Control) + (2 × Defence Strength) - (5 × Damage Taken)

The numerical weights are example values and can be adjusted according to the game.

Factors Considered:

1. **Resource Advantage:** Measures the resources available to the player for building units and improving the game position.

2. **Unit Strength:** Evaluates the number and strength of available military units.

3. **Territory Control:** Measures how much important territory the player controls.

4. **Defence Strength:** Evaluates the player's ability to protect important areas and units.

5. **Damage Taken:** Measures the losses suffered by the player.

A higher evaluation score indicates a more favourable position for the AI.

These factors help the AI compare different game situations and select better moves.

v. Recommend a strategy for balancing optimality and real-time performance.

I would recommend combining **Minimax, Alpha-Beta Pruning and Iterative Deepening Search**.

Minimax helps the AI evaluate possible moves and opponent responses.

Alpha-Beta Pruning reduces unnecessary exploration and saves processing time.

Iterative Deepening Search allows the AI to search at increasing depths while keeping track of the best move found so far.

For example, the AI can first search one level deep, then two levels and continue until its available time is almost over.

If the time limit is reached, it can use the best move found during the previous completed search.

Move ordering and a good evaluation function can further improve its performance.

This combination allows the AI to make intelligent decisions without spending too much time calculating every possible move.

Therefore, combining these techniques provides a good balance between decision quality, computational efficiency and real-time performance.